

Network Security

Theory Class Nº6

Jorge Sa Silva
sasilva@deec.uc.pt
University of Coimbra
Portugal

Outline

- Cryptography
  - Hash function
  - Diffie-Hellman
  - Transport Layer Security (TLS)



Cryptography



- Perspective:
  - User
  - Manager
  - Developer

Introduction to Cryptography



- Cryptography is the science of using mathematics to encrypt and decrypt data

- Cryptography is important:
  - Data integrity
  - User authentication
  - Information confidentiality

"A good cryptographic algorithm must be based on a balanced compromise between complexity and usability"

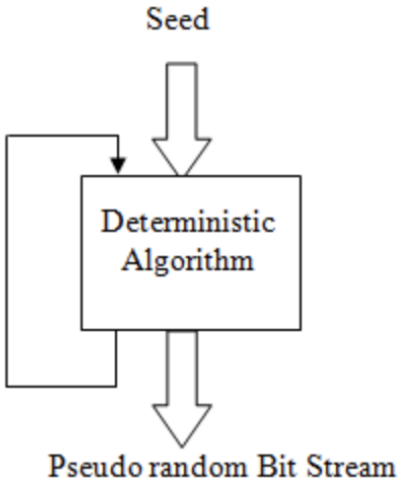
Kerckhoffs's principle

- The security of the encryption scheme must depend only on secrecy of the key and not on the secrecy of the algorithm
- As opposed to "security through obscurity"
- New ciphers and algorithms are not trusted without public scrutiny over years.



Some concepts

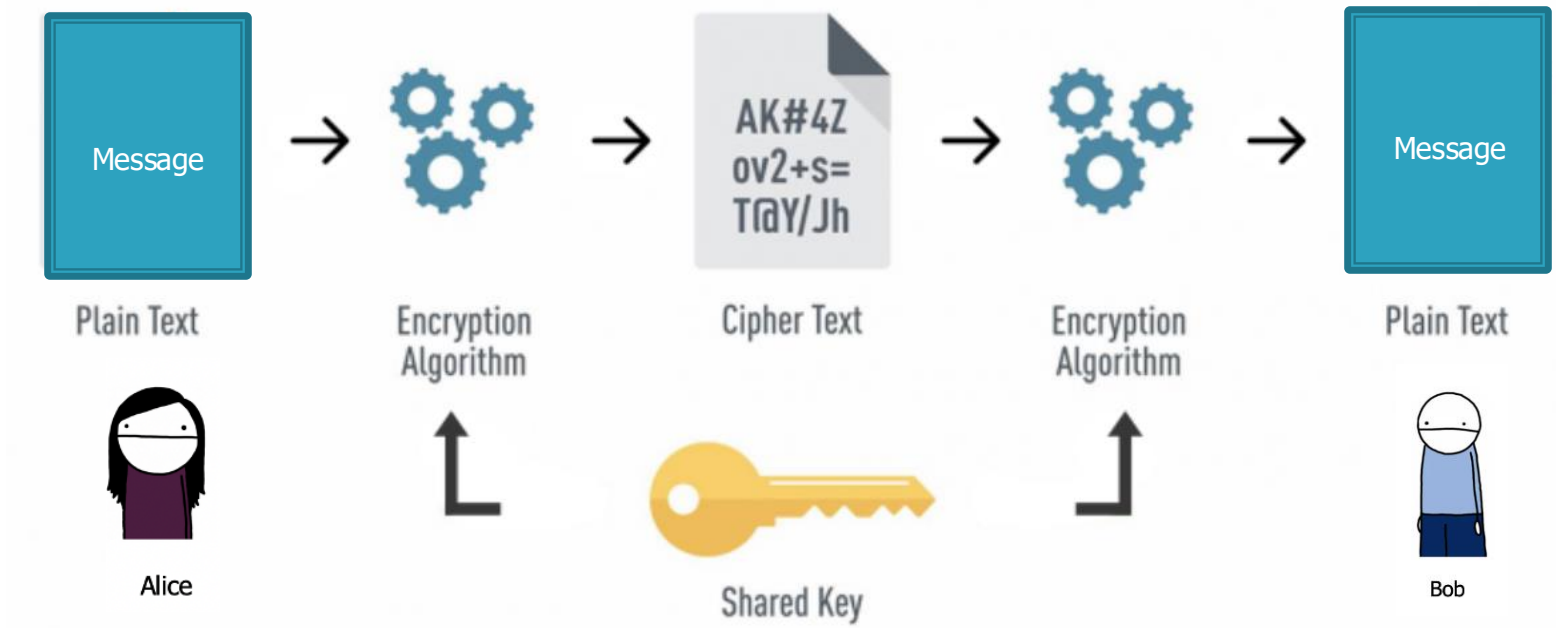
- One-way (irreversible) functions
  - Difficult to apply y=f(x)
  - Ex: m mod n = z
- Random and pseudo-random number generators
  - Problem: period
    - May not be a problem if the seed is not known and the period is large enough
  - Need to restart with (random) seeds



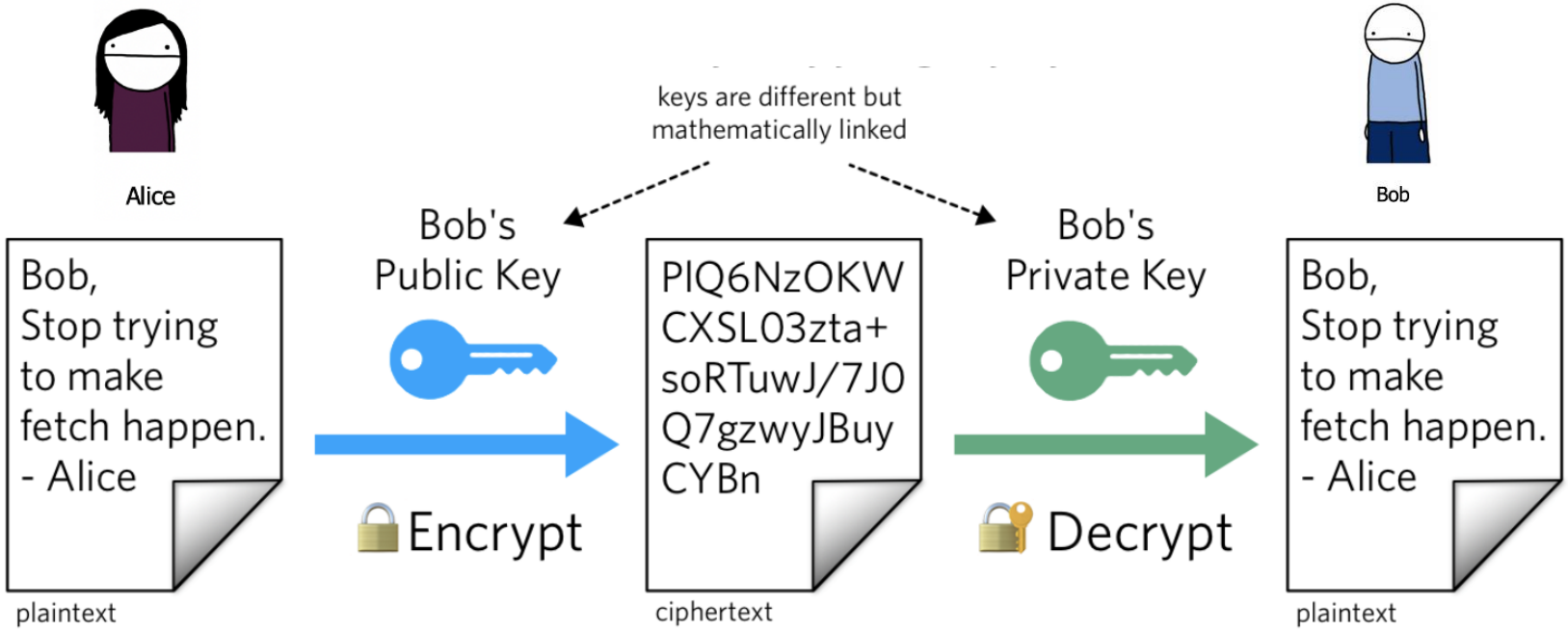
Encryption Strength

- Depends on:
  - Algorithm
  - The secrecy of the key
  - The length of the key
  - The initialization vectors
  - How all work together
- Work factor
  - An estimate of the effort and resources it would take to penetrate a cryptosystem
- Strength of protection mechanism should be based in correlation to the sensitivity of the data being encrypted.

Symmetric Encryption

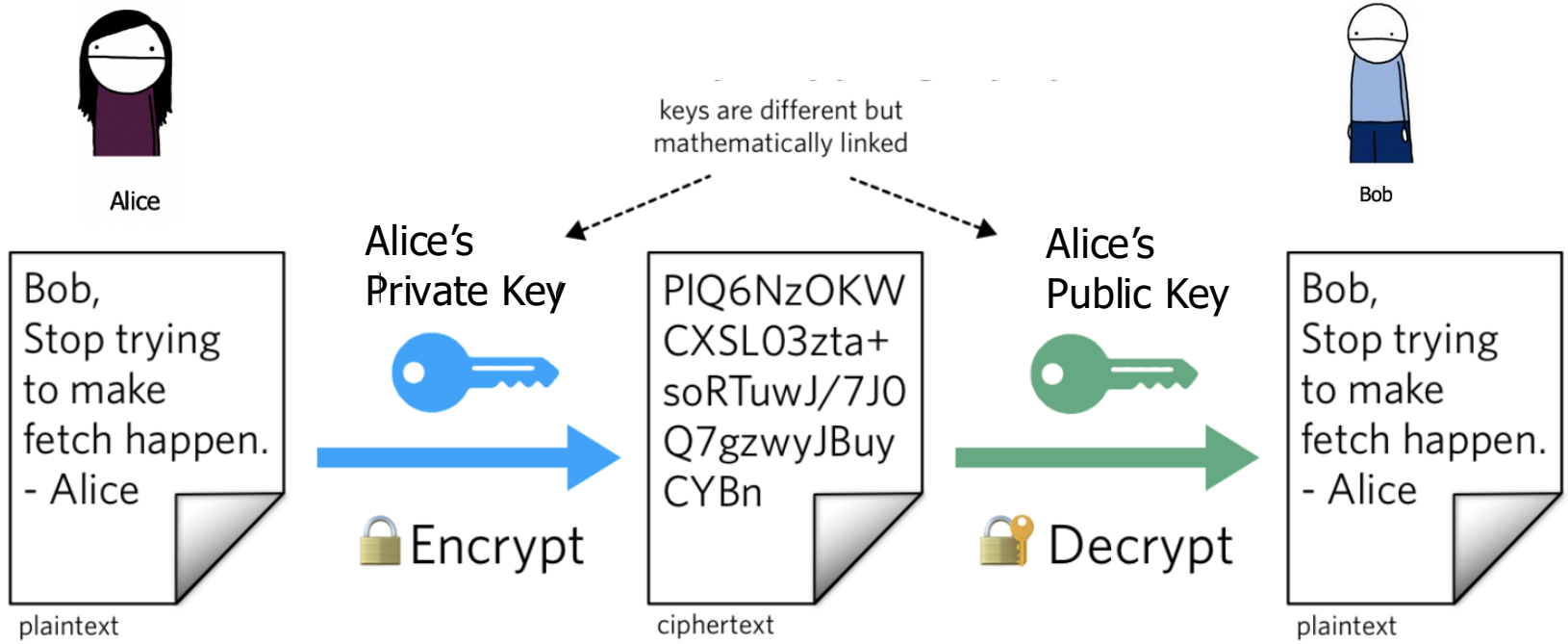


Asymmetric Encryption (confidentiality)



Only Bob can read the message!

Asymmetric Encryption (authentication)



Symmetric vs Asymmetric

- Asymmetric encryption requires more computational power
- Asymmetric encryption can be used to distribute symmetric keys
- Symmetric keys should be reset from time to time

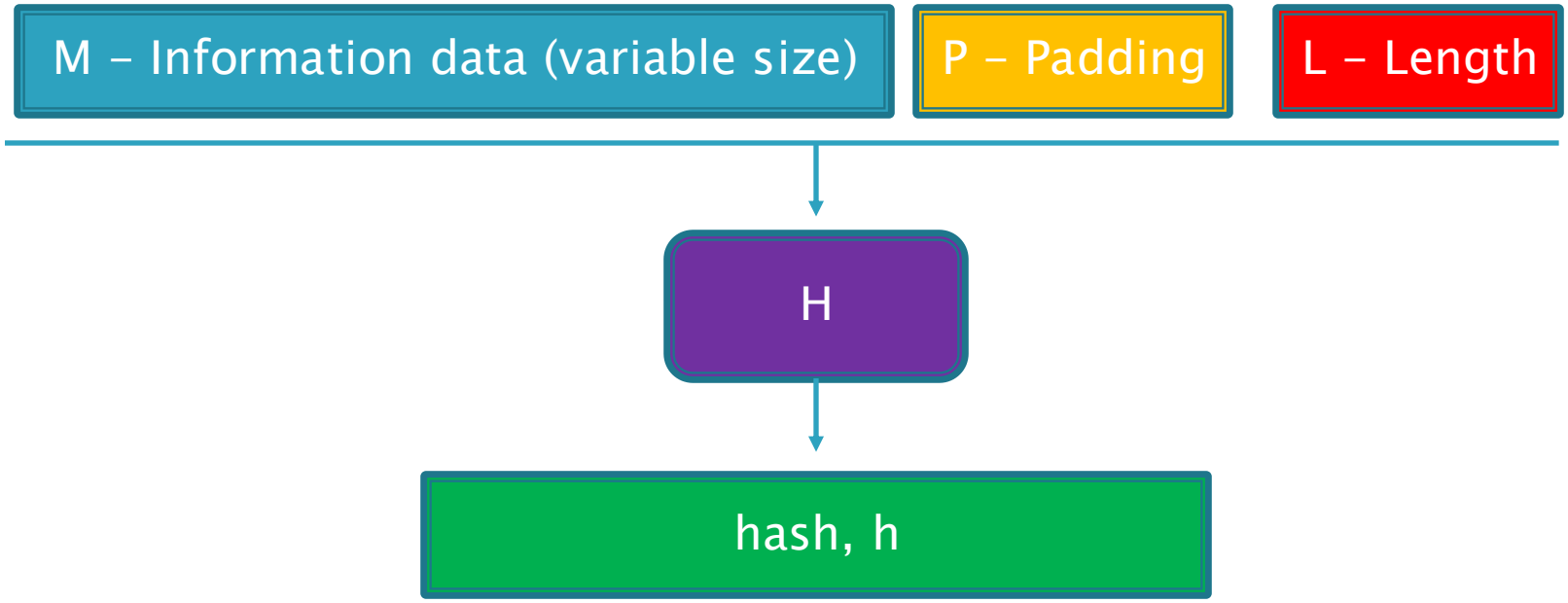
Cryptanalysis and Brute – Force Attacks

- Objective of attacker: recover key (not just message)
- Approaches of attacker:
  - Cryptanalysis: Exploit characteristics of algorithm to deduce plaintext or key
  - Brute-force attack: Try every possible key on ciphertext until intelligible translation into plaintext obtained
- Important note: if attack succeeds, all future/past messages are compromised



Symmetric Encryption

HASH functions



- A hash function usually means a function that compresses, meaning the output is shorter than the input.
- Padding is necessary as the input is usually of fixed size. L – to know the length of M
- The hash value is representative of the original input.
- A hash function can be used to create a finger printing. It 's almost impossible to alter the file without altering its hash value.

Hash Functions

The ideal cryptographic hash function has four main properties:

- it is easy to compute the hash value for any given message
- it is infeasible to generate a message that has a given hash
- it is infeasible to modify a message without changing the hash
- it is infeasible to find two different messages with the same hash.

HASH functions

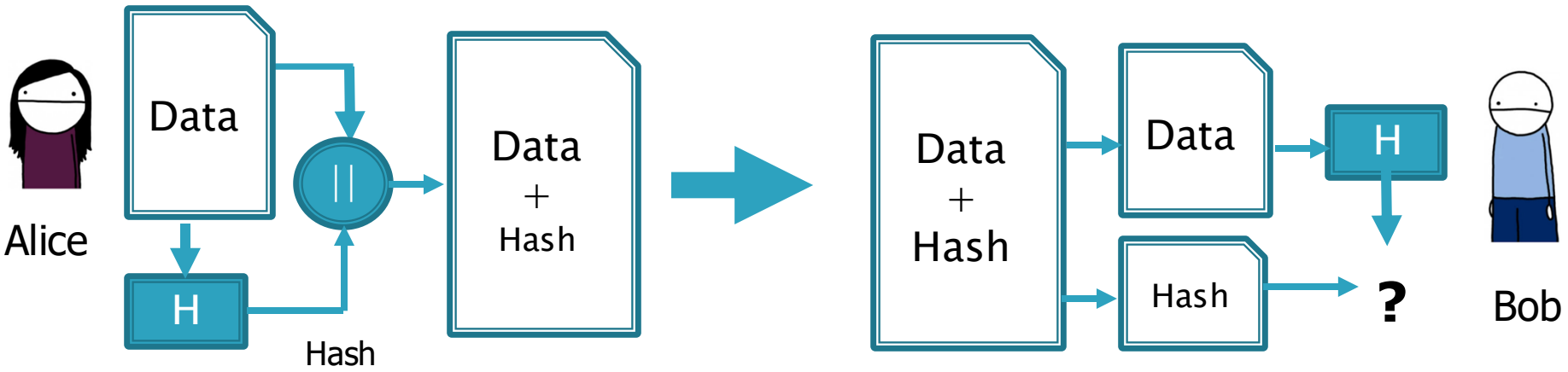
- Main objective: Data Integrity
- h = H(M)
- Size (M || P || L) = k*b

b – base value
k – integer value

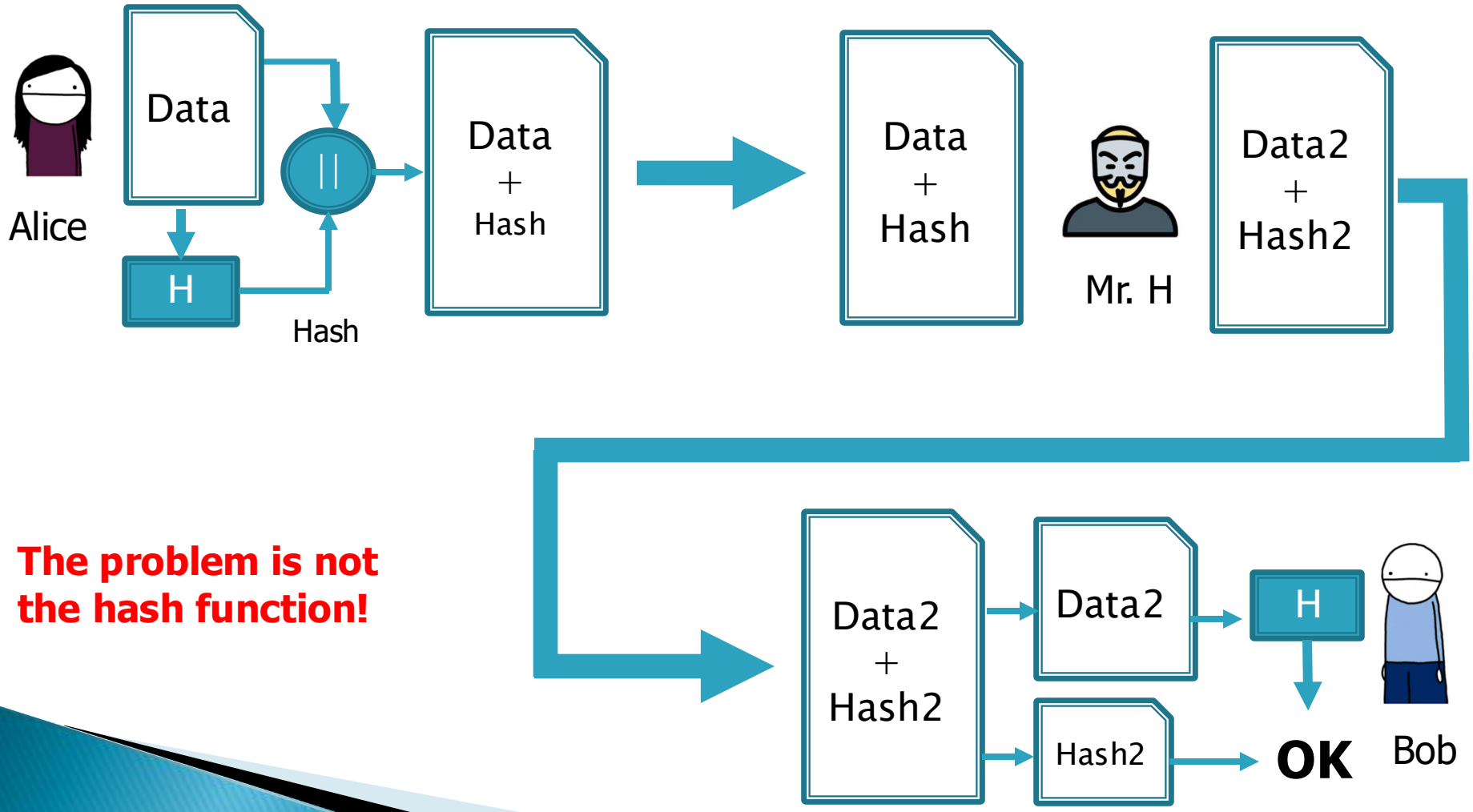
HASH functions

Requirement	Description
Variable input size	H can be applied to a block of data of any size.
Fixed output size	H produces a fixed-length output.
Efficiency	$H(x)$ is relatively easy to compute for any given x, making both hardware and software implementations practical.
Preimage resistant (one-way property)	For any given hash value h, it is computationally infeasible to find y such that $H(y) = h$.
Second preimage resistant (weak collision resistant)	For any given block x, it is computationally infeasible to find $y \neq x$ with $H(y) = H(x)$.
Collision resistant (strong collision resistant)	It is computationally infeasible to find any pair (x, y) with $x \neq y$, such that $H(x) = H(y)$.
Pseudorandomness	Output of H meets standard tests for pseudorandomness.

Applications of Hash functions – Integrity



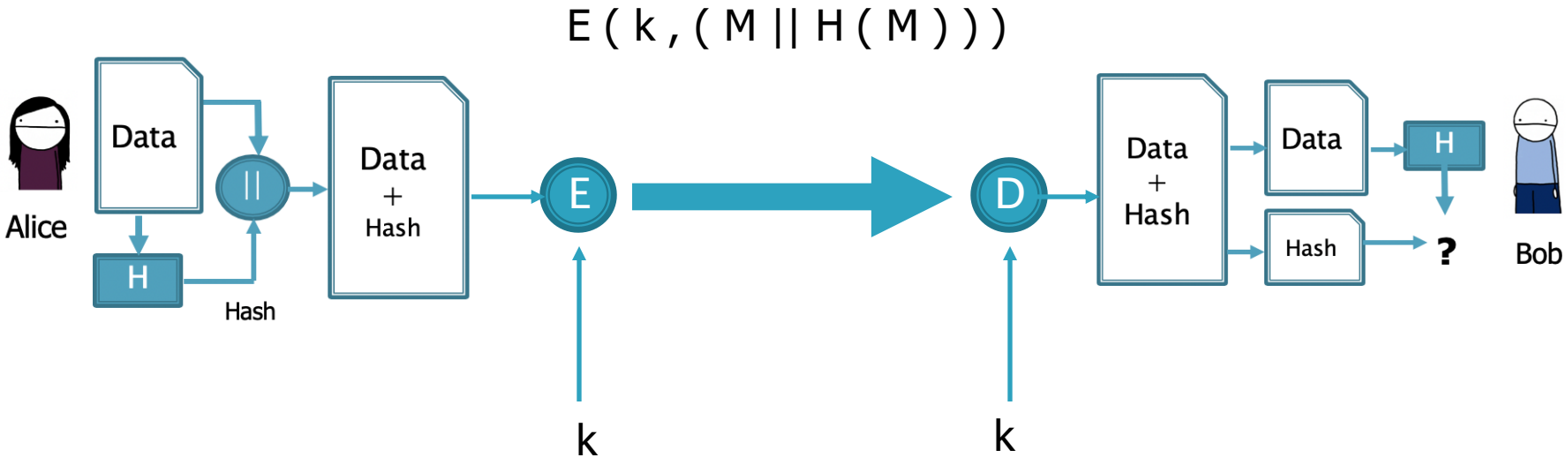
Man-in-the-middle attack



The problem is not the hash function!

Applications of Hash functions

Integrity + Authentication + Confidentiality

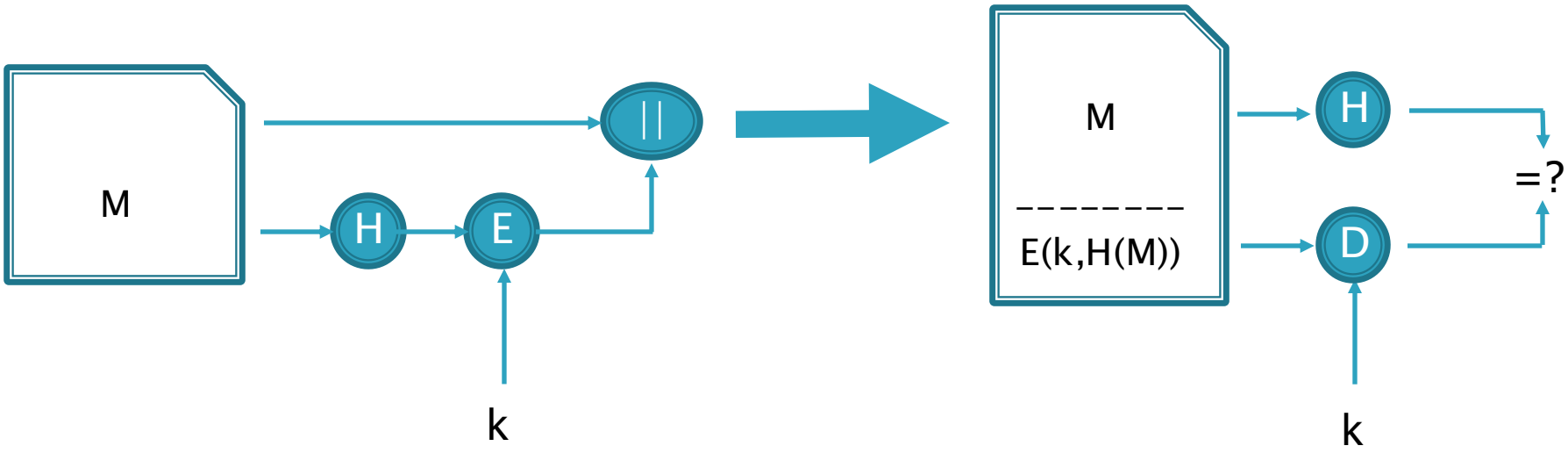


K – shared key

Applications of Hash functions

Integrity + Authentication

- Confidentiality not necessary

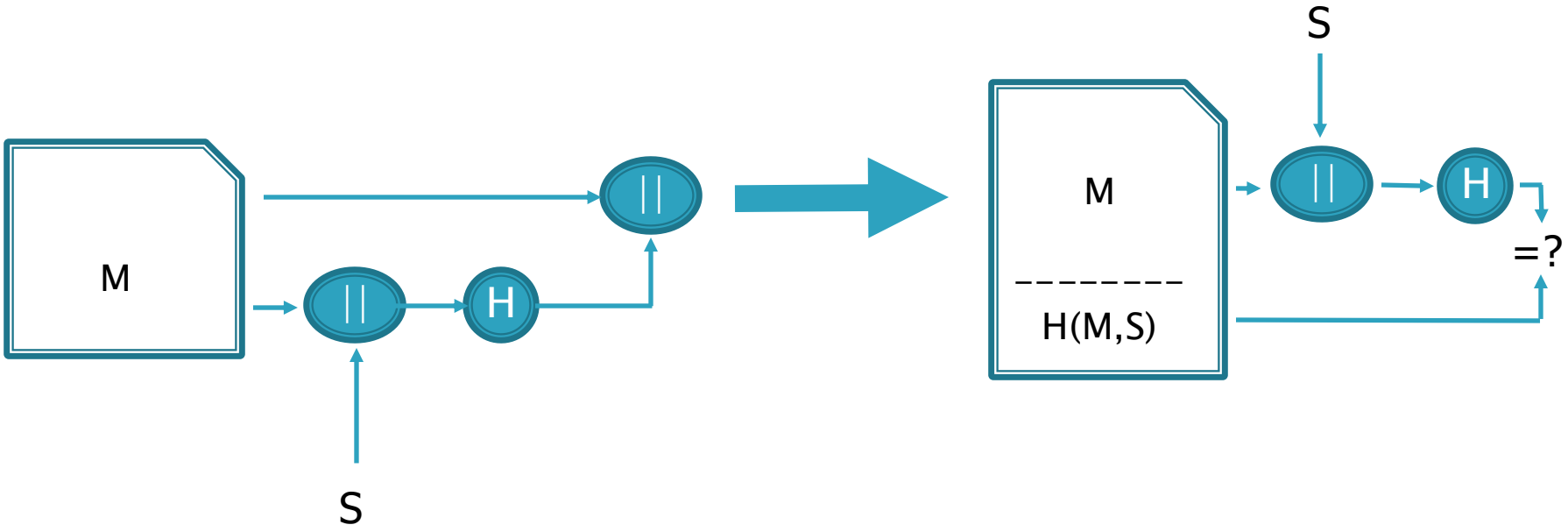


K – shared key

Applications of Hash functions

Integrity + Authentication

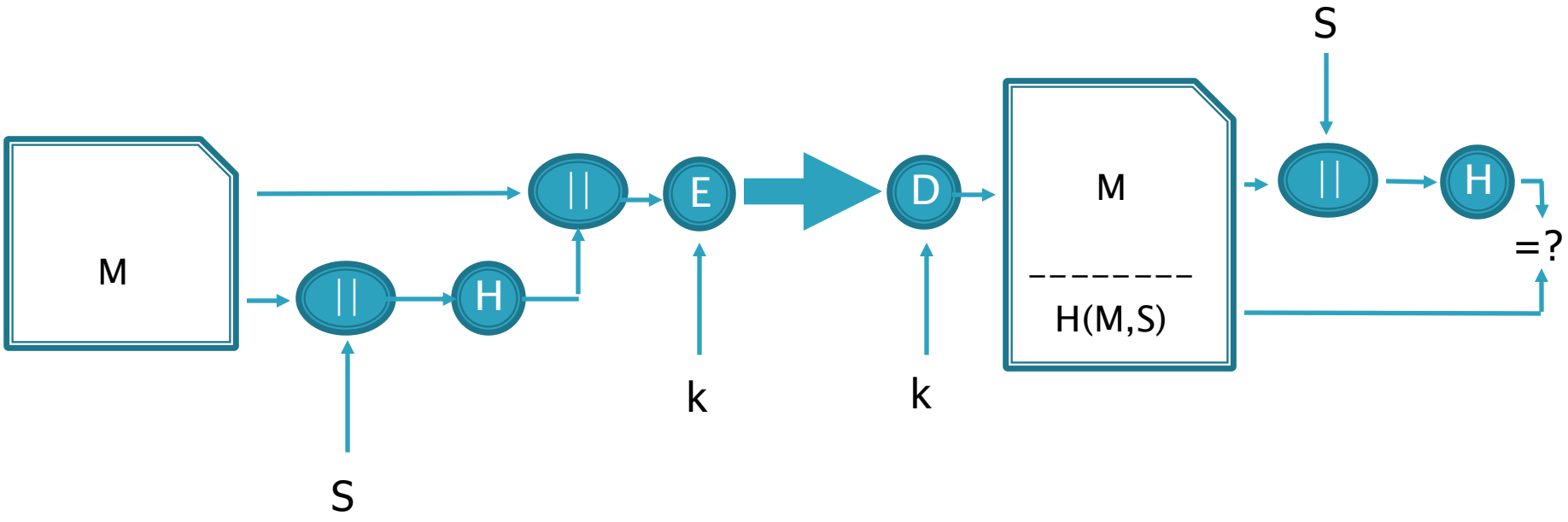
- Confidentiality not necessary
- Use of secret code (S) instead of encryption/decryption



Applications of Hash functions

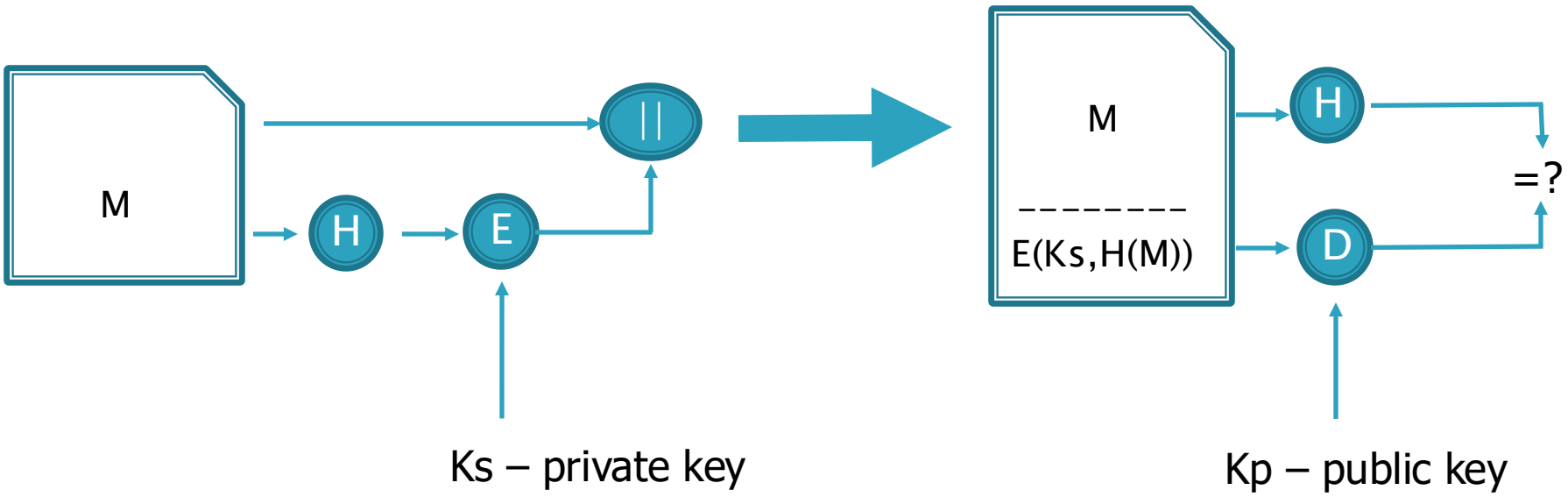
Integrity + Authentication + Confidentiality

- Use of secret code (S) and encryption(E) / decryption(D)

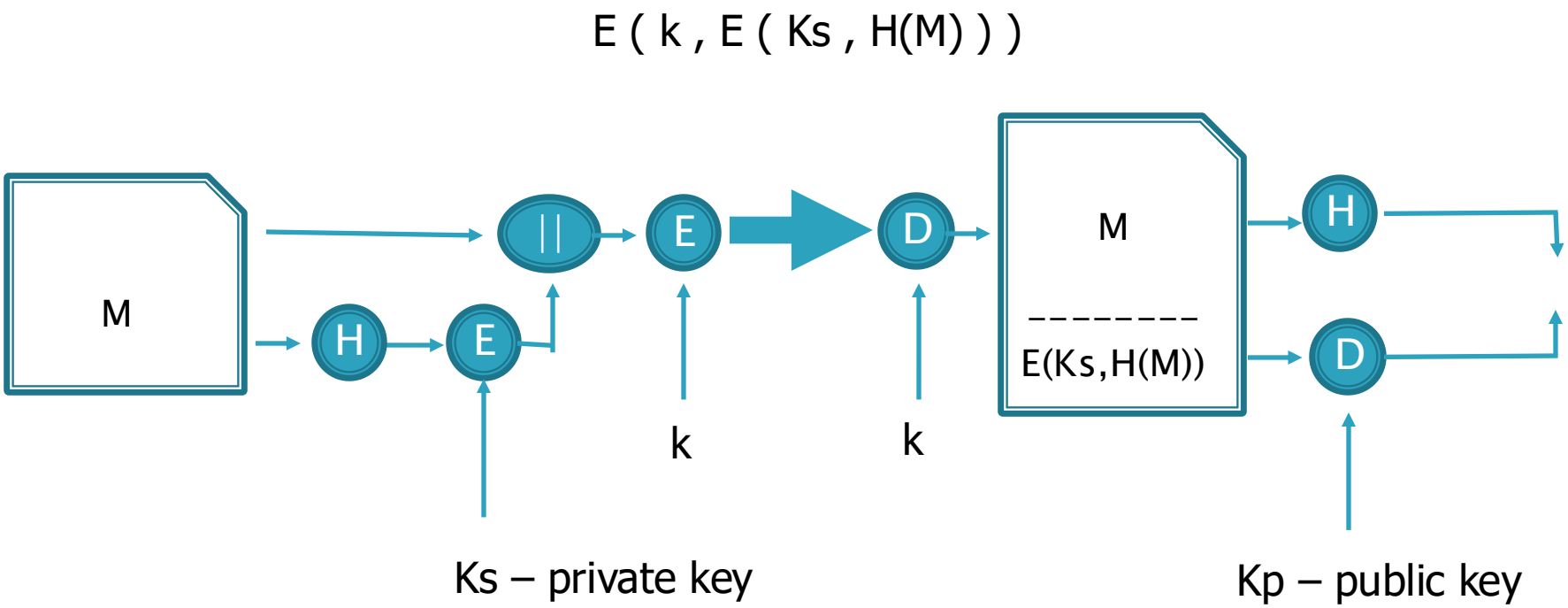


K – shared key

Applications of Hash functions
Digital signatures
Integrity + Authentication



Applications of Hash functions
Digital signatures
Integrity + Authentication + Confidentiality



Hash functions – other applications

- Files with passwords
  - Usually OS do not save password but use their hash codes
- Intrusion detection in files
- Virus detection
- Generation of pseudorandom numbers

Hash functions – examples (1)

$$C_i = b_{i1} \oplus b_{i2} \oplus \ldots \oplus b_{im}$$

C_i = ith bit of the hash code, $1 \leq i \leq$ n
m = number of n-bit blocks in the input
b_{ij} = ith bit in jth block
$\oplus$ = XOR operation

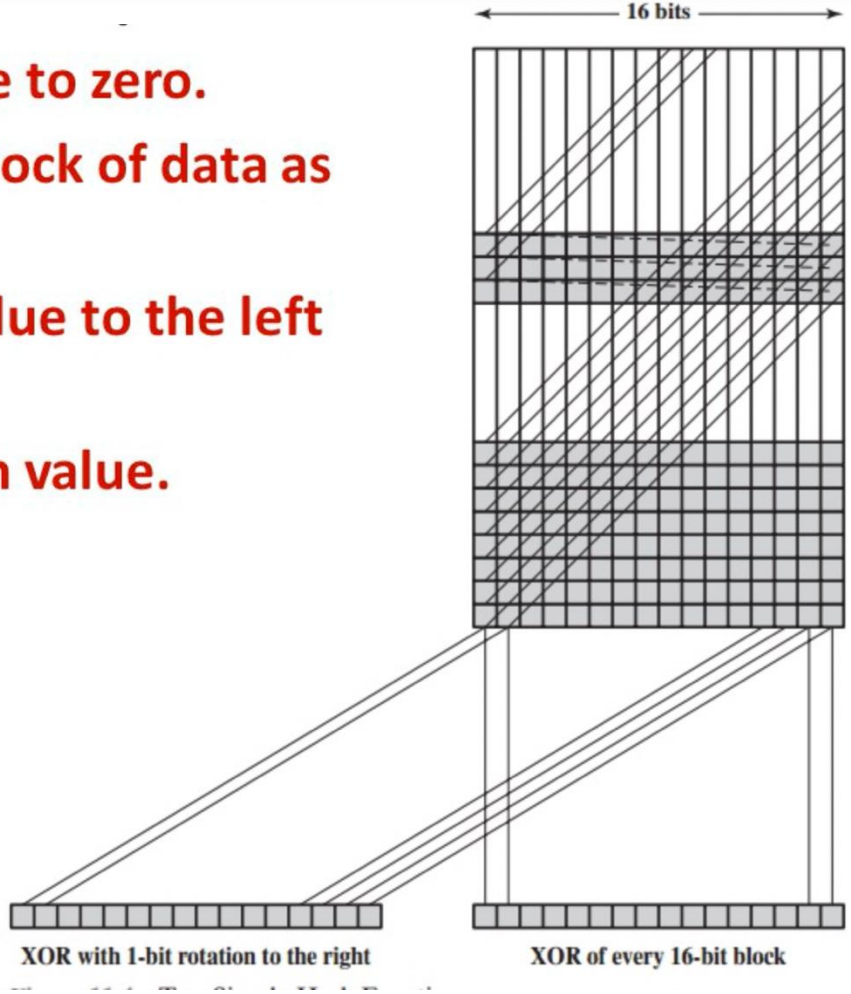
Hash functions – examples (2)

i. Initially set the -bit hash value to zero.

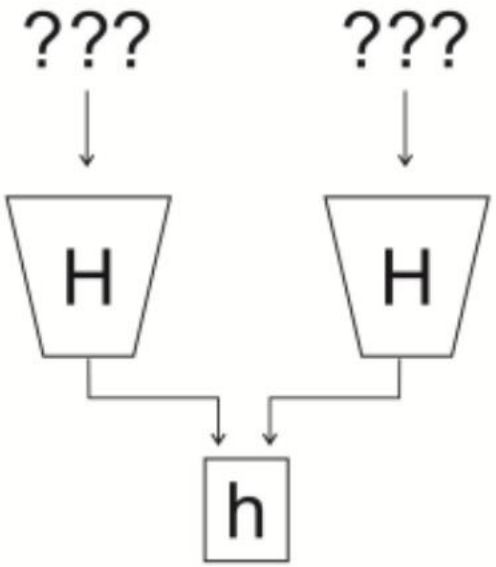
ii. Process each successive -bit block of data as follows:

a. Rotate the current hash value to the left by one bit.

b. XOR the block into the hash value.



Collision Resistance



Well Known Hash Functions

- MD5
  - output 128 bits
  - collision resistance completely broken by researchers in China in 2004
- SHA1 (Secure Hash Algorithm)
  - output 160 bits
  - no collision found yet, but method exist to find collisions in less than 2^80
  - considered insecure for collision resistance
  - one-wayness still holds
- SHA2 (SHA-224, SHA-256, SHA-384, SHA-512)
  - outputs 224, 256, 384, and 512 bits, respectively
  - No real security concerns yet

Well Known Hash Functions

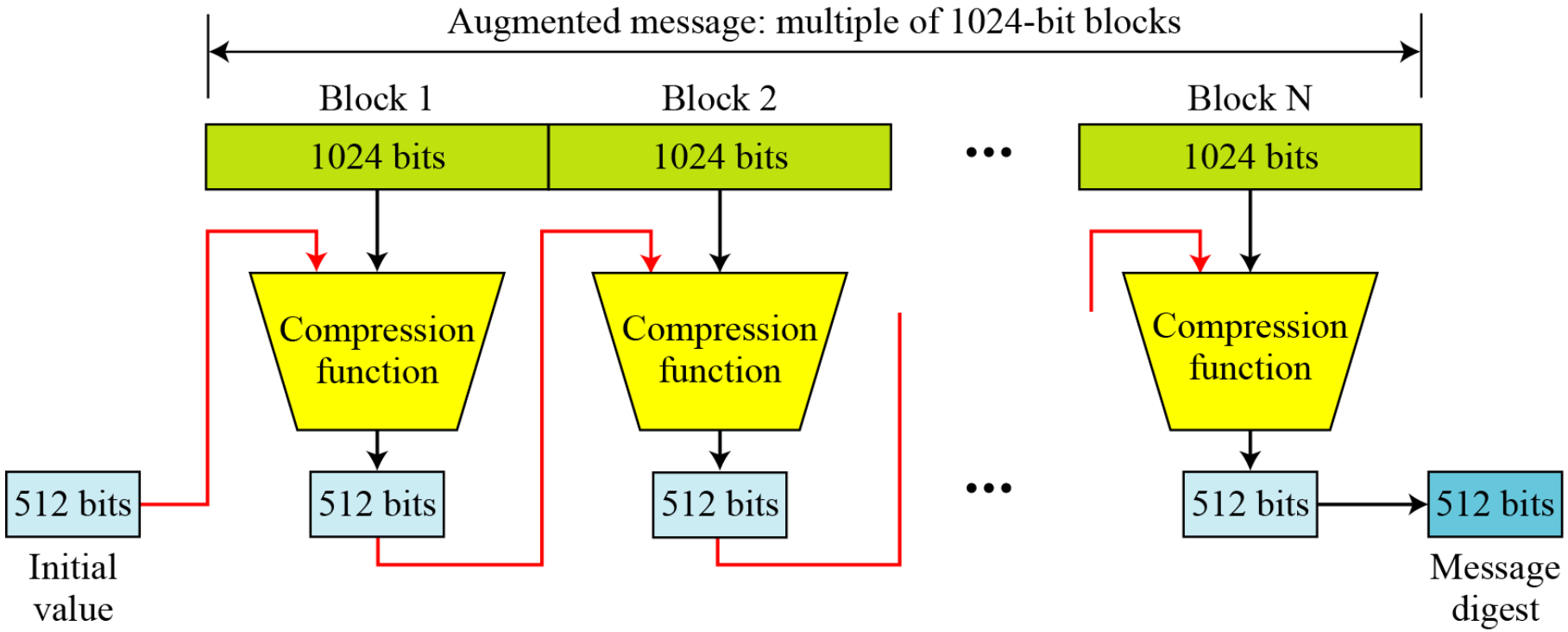
Algorithm	Message Size (bits)	Block Size (bits)	Word Size (bits)	Message Digest Size (bits)	Operations
SHA-1	< 2^64	512	32	160	+,and,or,xor,rot
SHA-256	< 2^64	512	32	256	+,and,or,xor,shr,rot
SHA-384	< 2^128	1024	64	384	+,and,or,xor,shr,rot
SHA-512	< 2^128	1024	64	512	+,and,or,xor,shr,rot

Sample Processing

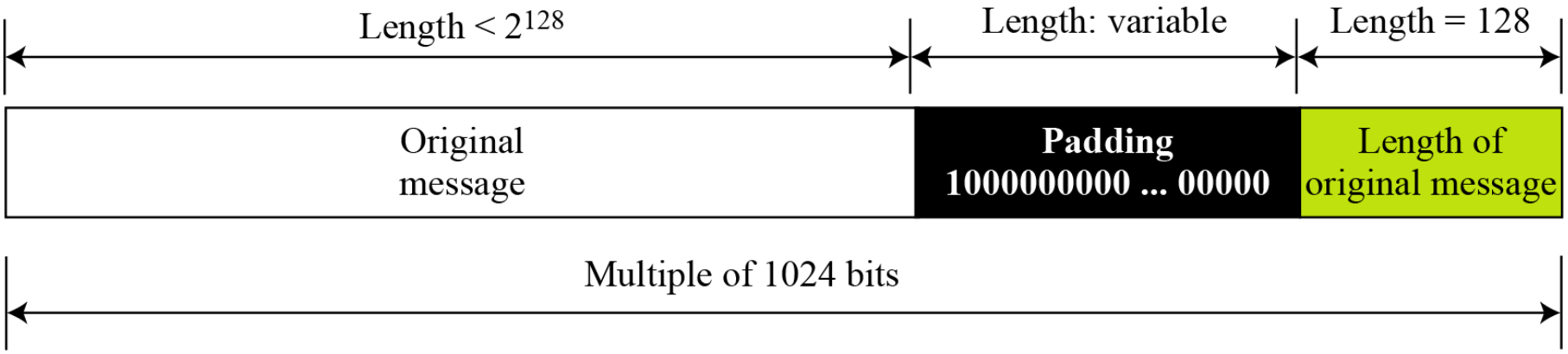
Type	bits	data processed
MD5	128	469.7 MB/s
SHA-1	160	339.4 MB/s
SHA-512	512	177.7 MB/s

- Mac Intel 2.66 Ghz core i7
- 1024 bytes block of data

SHA-512 Overview



Padding and length field in SHA-512

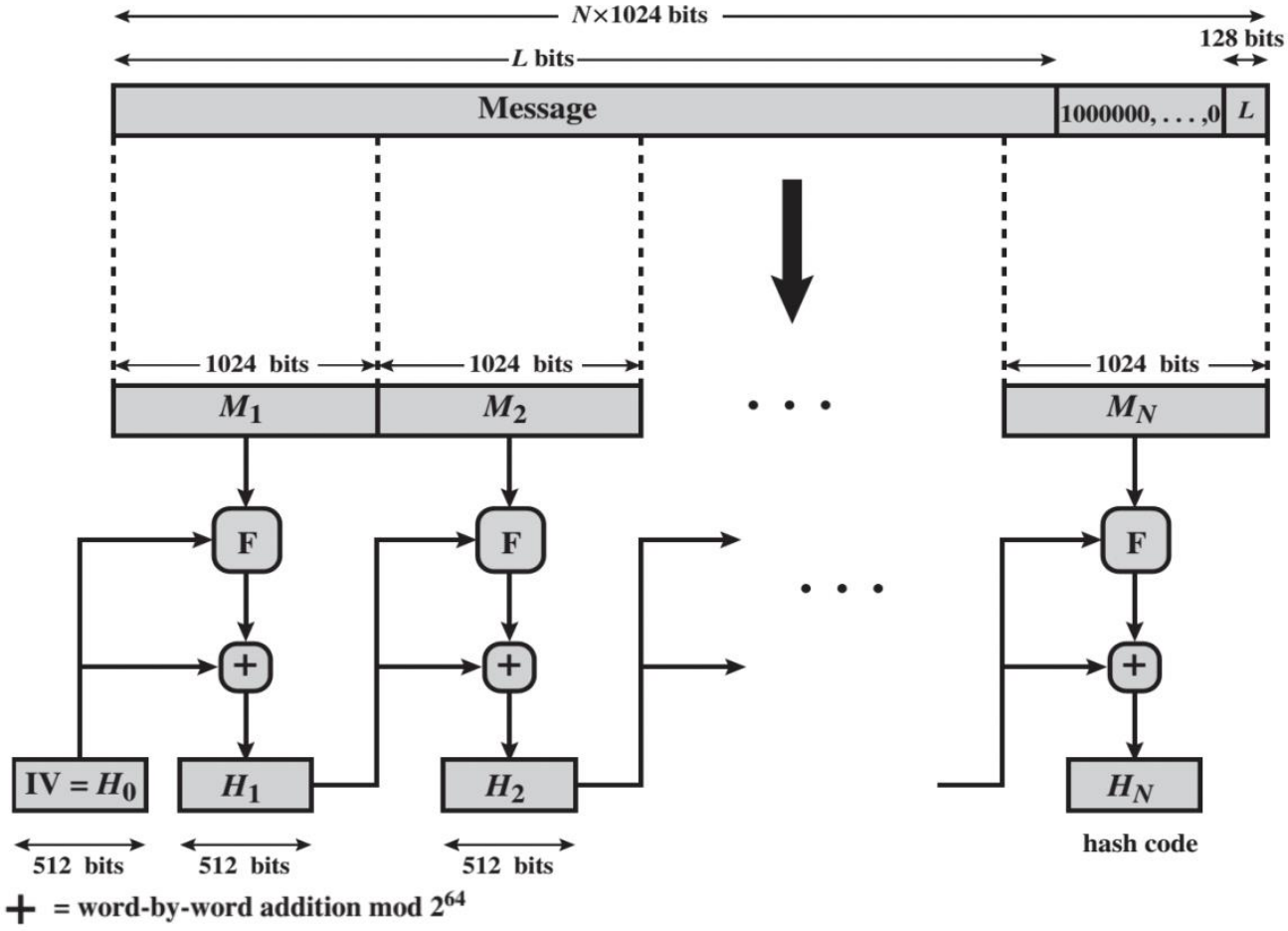


- **What is the number of padding bits if the length of the original message is 2590 bits?**
- We can calculate the number of padding bits as follows:

$$|P| = (-2590 - 128) \bmod 1024 = -2718 \bmod 1024 = 354$$

- The padding consists of one 1 followed by 353 0's.

SHA-512 Overview



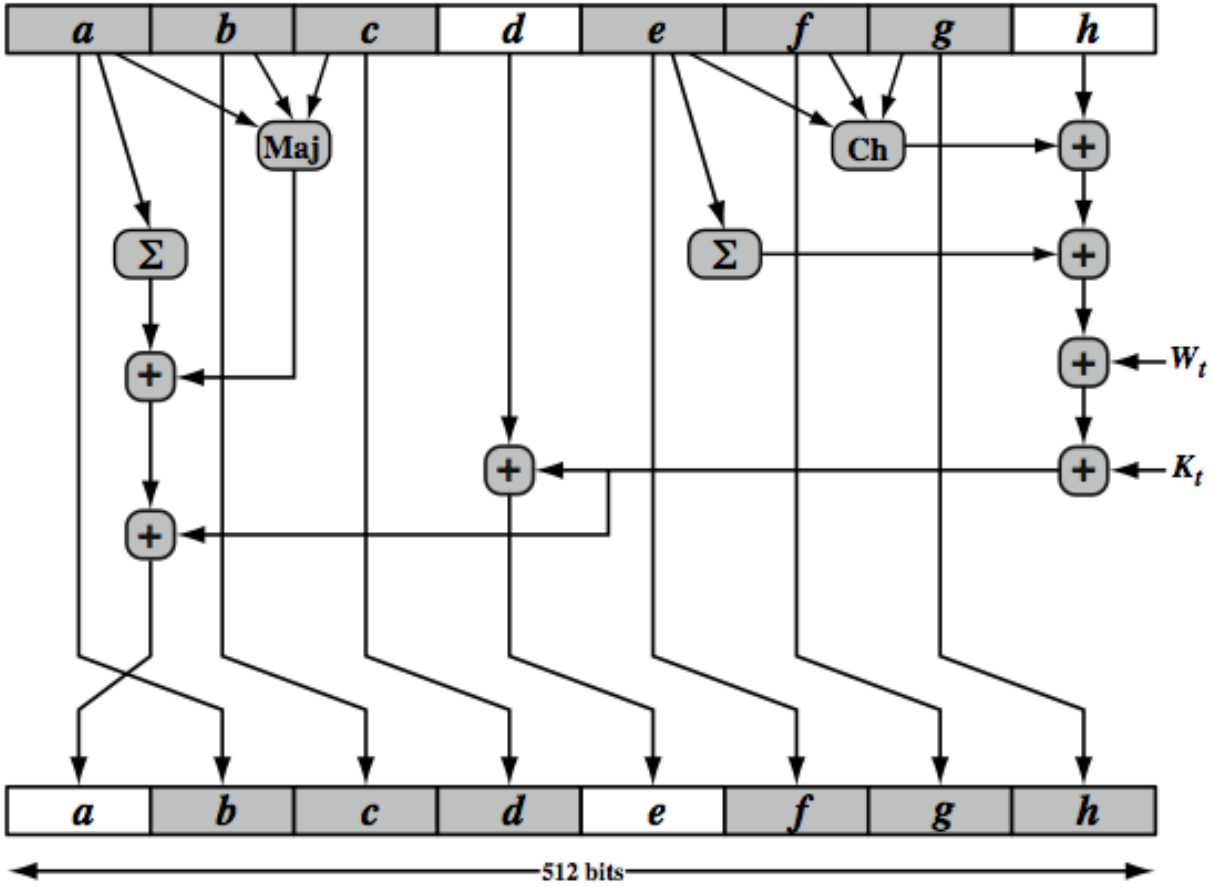
H0

Buffer	Value (in Hexadecimal)	Buffer	Value (in Hexadecimal)
A_0	6A09E667F3BCC908	E_0	510E527FADE682D1
B_0	BB67AE8584CAA73B	F_0	9B05688C2B3E6C1F
C_0	3C6EF372FE94F82B	G_0	1F83D9ABFB41BD6B
D_0	A54FF53A5F1D36F1	H_0	5BE0CD19137E2179

K0 ... K79

428A2F98D728AE22	7137449123EF65CD	B5C0FBCFEC4D3B2F	E9B5DBA58189DBBC
3956C25BF348B538	59F111F1B605D019	923F82A4AF194F9B	AB1C5ED5DA6D8118
D807AA98A3030242	12835B0145706FBE	243185BE4EE4B28C	550C7DC3D5FFB4E2
72BE5D74F27B896F	80DEB1FE3B1696B1	9BDC06A725C71235	C19BF174CF692694
E49B69C19EF14AD2	EFBE4786384F25E3	0FC19DC68B8CD5B5	240CA1CC77AC9C65
2DE92C6F592B0275	4A7484AA6EA6E483	5CB0A9DCBD41FBD4	76F988DA831153B5
983E5152EE66DFAB	A831C66D2DB43210	B00327C898FB213F	BF597FC7BEEF0EE4
C6E00BF33DA88FC2	D5A79147930AA725	06CA6351E003826F	142929670A0E6E70
27B70A8546D22FFC	2E1B21385C26C926	4D2C6DFC5AC42AED	53380D139D95B3DF
650A73548BAF63DE	766A0ABB3C77B2A8	81C2C92E47EDAEE6	92722C851482353B
A2BFE8A14CF10364	A81A664BBC423001	C24B8B70D0F89791	C76C51A30654BE30
D192E819D6EF5218	D69906245565A910	F40E35855771202A	106AA07032BBD1B8
19A4C116B8D2D0C8	1E376C085141AB53	2748774CDF8EEB99	34B0BCB5E19B48A8
391C0CB3C5C95A63	4ED8AA4AE3418ACB	5B9CCA4F7763E373	682E6FF3D6B2B8A3
748F82EE5DEFB2FC	78A5636F43172F60	84C87814A1F0AB72	8CC702081A6439EC
90BEFFFA23631E28	A4506CEBDE82BDE9	BEF9A3F7B2C67915	C67178F2E372532B
CA273ECEEA26619C	D186B8C721C0C207	EADA7DD6CDE0EB1E	F57D4F7FEE6ED178
06F067AA72176FBA	0A637DC5A2C898A6	113F9804BEF90DAE	1B710B35131C471B
28DB77F523047D84	32CAAB7B40C72493	3C9EBE0A15C9BEBC	431D67C49C100D4C
4CC5D4BECB3E42B6	4597F299CFC657E2	5FCB6FAB3AD6FAEC	6C44198C4A475817

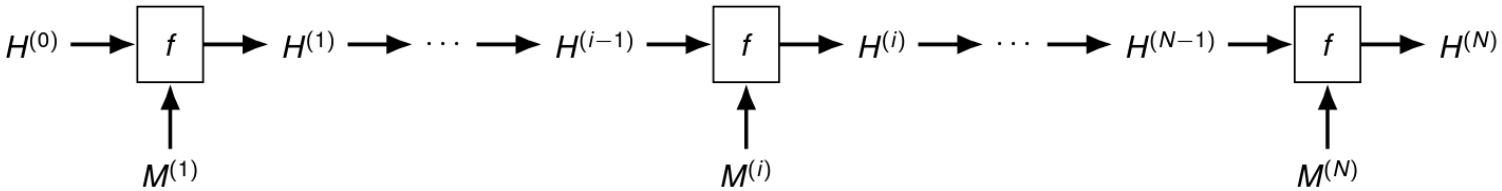
SHA-512 Round Function



Secure Hash Algorithm: SHA-256

1. Padded input is split into N 512-bit blocks $M^{(1)}, M^{(2)}, \ldots, M^{(N)}$
2. Given $H^{(i-1)}$, the next $H^{(i)}$ is calculated using a function f

$$H^{(i)} = f(M^{(i)}, H^{(i-1)}), \quad 1 \leq i \leq N.$$



3. f is called a *compression function*
4. $H^{(N)}$ is the output of SHA-256 for input M

NIST SHA-3 Competition

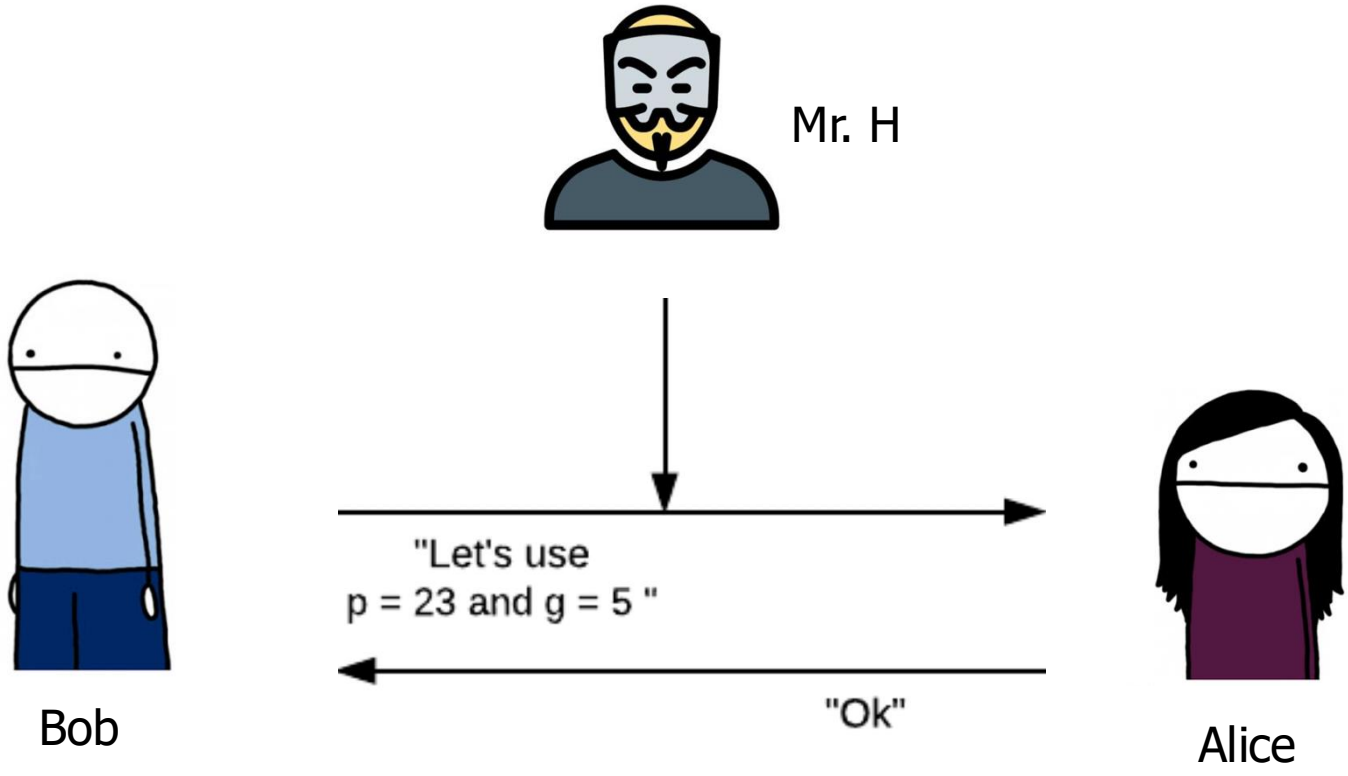
- NIST is having an ongoing competition for SHA-3, the next generation of standard hash algorithms
- 2007: Request for submissions of new hash functions
- 2008: Submissions deadline. Received 64 entries. Announced first-round selections of 51 candidates.
- 2009: After First SHA-3 candidate conference in Feb, announced 14 Second Round Candidates in July.
- 2010: After one year public review of the algorithms, hold second SHA-3 candidate conference in Aug. Announced 5 Third-round candidates in Dec.
- 2011: Public comment for final round
- 2012: October 2, NIST selected SHA3
  - Keccak (pronounced “catch-ack”) created by Guido Bertoni, Joan Daemen and Gilles Van Assche, Michaël Peeters

Diffie-Hellman-Merkle



- **Diffie-Hellman-Merkle is a way to share a secret key with someone (or something) without actually sending them the key.**
  - U.S. Patent 4,200,770, from 1977

Diffie-Hellman (step 1)



Diffie-Hellman (step 2)

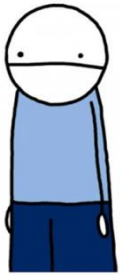


Mr. H



Alice

4



Bob

2. Alice needs to calculate a private key.

She does this by picking a secret number (a). She computes $G^a \ mod \ p$ and sends that result to Bob.

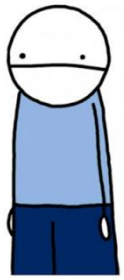
Alice chooses a secret, random integer a = 4.

Alice computes $A = 5^4 \ mod \ 23 = 4$ and sends the number 4 to Bob.

Diffie-Hellman (step 3)



Mr. H



Bob

10



Alice

3. Bob makes his own private key.

He calculates this by picking a secret number (b) and computing $g^b \ mod \ p$. He then sends the result to Alice. Bob creates a random private key, for this example we'll use 3.

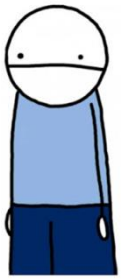
Then Bob calculates $B = 5^3 \ mod \ 23 = 10$ and sends 10 to Alice.

Diffie-Hellman (step 4)



Mr. H

Bob





Alice

4. Now Bob takes the number Alice sent him and computes $b^a \ mod \ p$.

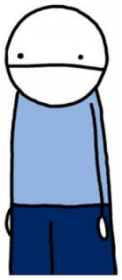
Bob computes $s = 4^3 \ mod \ 23 \ = 18$

Diffie-Hellman (step 5)



Mr. H

Bob





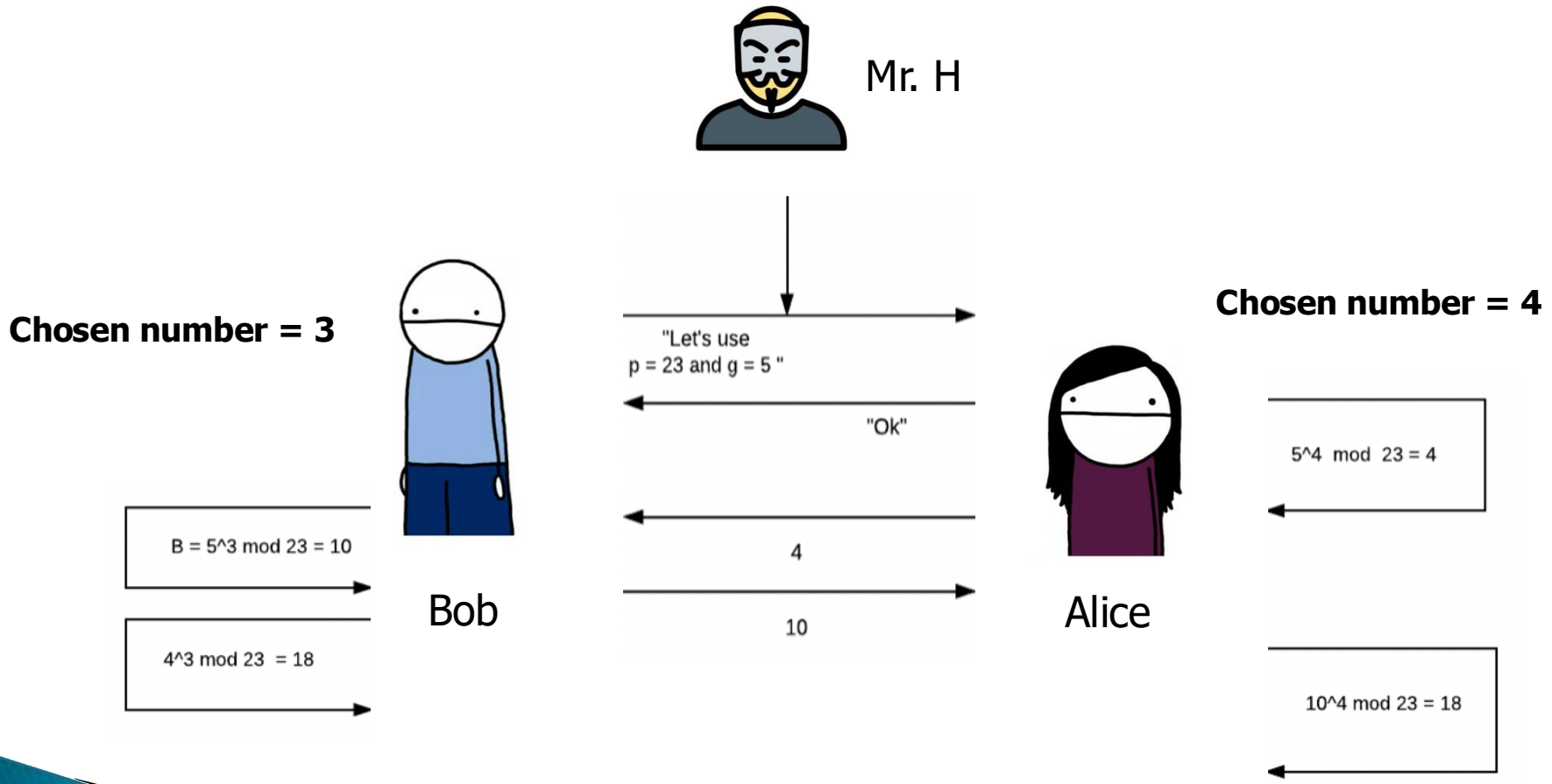
Alice

5. Alice computes $A^b \ mod \ p$.

Alice calculates $s = 10^4 \ mod \ 23 = 18$

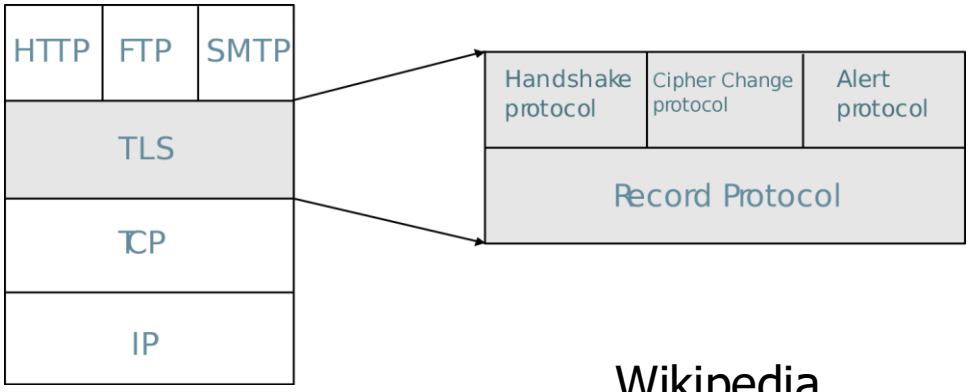
The magic is that Alice and Bob now have the same number,

Diffie-Hellman (all steps)



Transport Layer Security (TLS)

- TLS is the successor of the now-deprecated Secure Sockets Layer (SSL)
- TLS is a cryptographic protocol designed to provide communications security over a computer network.
- The protocol is widely used in applications such as email, instant messaging, and voice over IP, but its use in securing HTTPS remains the most publicly visible.



Wikipedia

History of TLS

- SSL 1.0
  - Internal Netscape design, early 1994?
  - Lost in the mists of time
- SSL 2.0
  - Published by Netscape, November 1994
  - Badly broken
- SSL 3.0
  - Designed by Netscape and Paul Kocher, November 1996
- TLS 1.X
  - Internet standard based on SSL 3.0, January 1999
  - Not interoperable with SSL 3.0
  - Current version: TLS 1.3 defined in RFC 8446

Transport Layer Security (TLS)

- **Confidentiality**: Encrypts data to prevent unauthorized access.
- **Integrity**: Ensures data isn't altered during transit.
- **Authentication**: Verifies the identity of the server (and optionally the client) using certificates.

Wikipedia



1) Handshake Process:

- **Initiation**: When a client (e.g., your browser) connects to a server (e.g., a website), it starts the TLS handshake. The client sends a "ClientHello" message, specifying supported TLS versions and cryptographic algorithms (cipher suites).
- **Server Response**: The server responds with a "ServerHello" message, selecting a compatible TLS version and cipher suite. It also sends its **digital certificate**, which contains its **public key** and is issued by a trusted Certificate Authority (CA).
- **Authentication**: The client verifies the server's certificate by checking it against trusted CAs to ensure the server is legitimate.
- **Key Exchange**: The client and server use a key exchange algorithm (e.g., Diffie-Hellman or RSA) to securely agree on a **symmetric session key**. This key will be used for encrypting data during the session. The process ensures that even if someone intercepts the handshake, they can't derive the session key.
- **Completion**: Both sides confirm the handshake is complete, and the secure connection is established.

2) Data Encryption:

- Once the handshake is done, all data exchanged is encrypted using the symmetric session key (typically using algorithms like AES). Symmetric encryption is fast and efficient for bulk data transfer.
- TLS also ensures **data integrity** using message authentication codes (MACs), which detect any tampering with the data.

3) Session Communication:

- The client and server now communicate securely, with all data encrypted and protected from eavesdropping or alteration. For example, when you visit an HTTPS website, TLS ensures your passwords, credit card details, or other sensitive data remain private.

4) Termination:

- When the session ends (e.g., you close the browser tab), the connection is terminated, and the session key is discarded. A new handshake is required for future connections.



Questions?